



Security Audit Report

Coinlist Pull 12

v1.0

September 21, 2026

Table of Contents

[Table of Contents](#)

[License](#)

[Disclaimer](#)

[Introduction](#)

[Purpose of This Report](#)

[Codebase Submitted for the Audit](#)

[Methodology](#)

[Functionality Overview](#)

[How to Read This Report](#)

[Code Quality Criteria](#)

[Summary of Findings](#)

[Detailed Findings](#)

- [1. Permanent denial of service of a \(user, input, output\) swap pair once the packed uint72/uint88 OptimizedSwapTotal accumulators saturate, as they are checked-add and have no reset path](#)
- [2. Sell slippage floor is enforced pre-fee and bps\[Se11\] is read at execution, so a fee change retroactively re-prices already-signed authorizations](#)
- [3. A whitelist entry can become permanently unremovable because Soms.setInputToken enforces its isContract and decimals\(\) guards on the removal path as well as the addition path](#)
- [4. The CoinList VERIFY_REQUEST digest does not commit to the Ondo quote, so one authorization admits any quote sharing \(side, asset, expiration\)](#)
- [5. Soms.pause accepts a raw bitmask that need not intersect SWAP_LEVEL, so plausible operator values report the contract as Paused while swaps stay fully live](#)
- [6. Value is stranded in the Ondo broker and a standing manager allowance is left behind, because neither input leg reconciles what the manager actually consumed](#)
- [7. Ondo.sol declares no status\(\) override, unlike its sibling venue, so the contract reports Active while its Ondo upstream is unusable and every swap reverts](#)
- [8. The Swapped event is not self-sufficient because the fee field carried by the sibling ITokenSwap event was deliberately dropped, so no on-chain quantity records what CoinList charged](#)
- [9. ondo/Types.sol:11 documents userId as binding the end user, while the deployed GMTokenManager pins it to the broker, so the comment conceals the complete absence of per-end-user quote binding](#)
- [10. The sole on-chain authorization root and the input-token whitelist are written without events, so signer rotation and the deployed whitelist are invisible to log-based monitoring](#)
- [11. Conventions carried forward from the pre-existing out-of-scope swap venues into Ondo/Soms: renounceOwnership, an irreversible stop\(\), a constructor-bypassed whitelist write, and an inert onlyOwner gate](#)

[Proof of Concept](#)

License



THIS WORK IS LICENSED UNDER A [CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE](https://creativecommons.org/licenses/by-nc/4.0/).

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED "AS IS", WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUCT ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

<https://oaksecurity.io>

info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by Amalgamated Token Services Inc. to perform a security audit of the Solidity smart contracts in src/swap in the <https://github.com/coinlist/onchain> repository.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	URL	Type	Commit
onchain	https://github.com/coinlist/onchain	Pull Request #12	Base: onchain@ffb7137091 Head: onchain@da33cdf99a Compare changes

For repositories reviewed as a Diff or Pull Request above, this audit covers only the changes introduced between the base and head references shown, and does not constitute a full review of the codebase at the head commit.

Scope	ethereum/ src/ swap/ ondo/ IManager.sol IRWA.sol Ondo.sol Types.sol ISoms.sol Soms.sol Types.sol
-------	--

Fixes Verified at Commit

These are the commits at which the client's fixes were reviewed. The status of each finding in this report (Resolved, Partially Resolved or Acknowledged) reflects the code at these commits.

fixes-review: [onchain@94ceb1c2a2](#)

Note that only fixes to the issues described in this report have been reviewed at this commit. Any further changes such as additional features have not been reviewed.

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
 - a. Race condition analysis
 - b. Under-/overflow issues
 - c. Key management vulnerabilities
4. Report preparation.

Functionality Overview

The repository contains Ethereum smart contracts developed by CoinList to support token sale and swap operations. The in-scope code is organized into two primary areas: a general swap module (`Soms.sol`, `ISoms.sol`, `Types.sol`) and an Ondo-specific swap module (`Ondo.sol`, `IManager.sol`, `IRWA.sol`, `Types.sol`) located under `ethereum/src/swap/`.

The Ondo module interfaces with external components via `IManager` and `IRWA`, suggesting integration with Ondo Finance's real-world asset infrastructure. Both modules define their own types, indicating distinct data structures for each swap context.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or losses, with medium to high probability.
Minor	A violation of best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects such as user experience or readability. This category may also include opinionated recommendations that the project team might not agree with.

The **status** of an issue can be one of the following: **Resolved**, **Acknowledged**, or **Partially Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	Medium	–
Code readability and clarity	Medium-High	Code is described by comments and architectural decisions are noted.
Level of documentation	Medium-High	Client provided extensive documentation.
Test coverage	Medium-High	Test coverage computed for the in scope files is 89.20%.

Summary of Findings

No	Description	Severity	Status
1	Permanent denial of service of a (user, input, output) swap pair once the packed uint72/uint88 OptimizedSwapTotal accumulators saturate, as they are checked-add and have no reset path	Minor	Resolved
2	Sell slippage floor is enforced pre-fee and bps[Sell] is read at execution, so a fee change retroactively re-prices already-signed authorizations	Minor	Acknowledged
3	A whitelist entry can become permanently unremovable because Soms.setInputToken enforces its isContract and decimals() guards on the removal path as well as the addition path	Minor	Resolved
4	The CoinList VERIFY_REQUEST digest does not commit to the Ondo quote, so one authorization admits any quote sharing (side, asset, expiration)	Minor	Acknowledged
5	Soms.pause accepts a raw bitmask that need not intersect SWAP_LEVEL, so plausible operator values report the contract as Paused while swaps stay fully live	Minor	Resolved
6	Value is stranded in the Ondo broker and a standing manager allowance is left behind, because neither input leg reconciles what the manager actually consumed	Minor	Partially Resolved
7	Ondo.sol declares no status() override, unlike its sibling venue, so the contract reports Active while its Ondo upstream is unusable and every swap reverts	Informational	Resolved
8	The Swapped event is not self-sufficient because the fee field carried by the sibling ITokenSwap	Informational	Partially Resolved

No	Description	Severity	Status
	event was deliberately dropped, so no on-chain quantity records what CoinList charged		
9	ondo/Types.sol:11 documents userId as binding the end user, while the deployed GMTokenManager pins it to the broker, so the comment conceals the complete absence of per-end-user quote binding	Informational	Resolved
10	The sole on-chain authorization root and the input-token whitelist are written without events, so signer rotation and the deployed whitelist are invisible to log-based monitoring	Informational	Partially Resolved
11	Conventions carried forward from the pre-existing out-of-scope swap venues into Ondo/Soms: renounceOwnership, an irreversible stop(), a constructor-bypassed whitelist write, and an inert onlyOwner gate	Informational	Resolved

Detailed Findings

1. Permanent denial of service of a (user, input, output) swap pair once the packed uint72/uint88 OptimizedSwapTotal accumulators saturate, as they are checked-add and have no reset path

Severity: Minor

- [ethereum/src/swap/Types.sol:18-25](#)
- [ethereum/src/swap/Soms.sol:34](#)
- [ethereum/src/swap/Soms.sol:47-49](#)
- [ethereum/src/swap/ondo/Ondo.sol:145](#)
- [ethereum/src/swap/ondo/Ondo.sol:147](#)
- [ethereum/src/swap/ondo/Ondo.sol:164](#)
- [ethereum/src/swap/ondo/Ondo.sol:189](#)
- [ethereum/src/swap/ondo/Ondo.sol:209](#)
- [ethereum/src/swap/ondo/Ondo.sol:211](#)
- [ethereum/src/swap/ISoms.sol:35-37](#)

Soms keeps lifetime, never-decreasing per-(user, inputToken, outputToken) swap totals in a packed struct of uint88 inputSum, uint72 feeSum, uint88 outputSum and uint8 count (ethereum/src/swap/Types.sol:20-25). Ondo writes the six accumulators through narrowing casts, each carrying a `// forge-lint: disable-next-line(unsafe-typecast)` suppression, at Ondo.sol:145, :147, :164 on Buy and :189, :209, :211 on Sell. The cast truncates the operand, but the `+=` targets the packed storage member, so under Solidity 0.8 the addition itself is checked. Saturation is not a silent wraparound: the swap reverts with `Panic(0x11)`, and does so on every subsequent attempt.

There is no reset path. `_totals` (Soms.sol:34) is internal, written only by `Ondo.buy/Ondo.sell` and read only by `totals()` (Soms.sol:47-49); no owner function clears or migrates it. A saturated slot permanently closes that pair for that user; the only escapes are a fresh EOA, a different whitelisted token, or a redeploy.

Under the shipped USDC-only configuration this is unreachable by roughly six orders of magnitude: with 6-decimal mainnet USDC (Ondo.sol:33) the binding uint72 feeSum caps at \$4.72e15 of cumulative fees. It activates on the first 18-decimal settlement token, which Soms.setInputToken permits (Soms.sol:91, decimals() <= 18). At 18 decimals the same ceiling is 4,722.37 whole tokens of fee: about \$560k of gross Buy volume at 85 bps (Ondo.sol:147) or \$944k redeemed at 50 bps (Ondo.sol:209), both the rates used in the project's own integration tests. feeSum is denominated in the input token on Buy and the output token on Sell, and neither the key nor the Swapped event records which.

Impact is denial of service, not loss: the transaction reverts whole, so no principal is stranded. It is self-inflicted only, never grievable, because `Ondo.sol:58` and `:68` key the slot on `msg.sender`, and the `CoinList` digest binds `req.sender = msg.sender (:61, :70)`.

→ [Proof of Concept: P.1](#)

Recommendation

We recommend widening the sums to `uint128` (one extra storage slot) or normalise stored values to a fixed precision such as 6-decimal USD, so the ceiling no longer depends on the whitelisted token's decimals; leave `uint8 count` as is, which is deliberate and documented (`Types.sol:19`). Add an owner-callable reset, or make the adds saturating, so a saturated slot is recoverable. Correct the `ISoms.sol:36` natspec to state, per side, which token each field is denominated in.

As hardening, consider folding `Side` into the key, e.g. `_totals[msg.sender][side][inputToken][outputToken]`. The key is currently direction-symmetric (`Ondo.sol:58` writes `[user][token][quote.asset]`, `:68` writes `[user][quote.asset][token]`), and keying on `Side` also removes the mixed-denomination `feeSum` ambiguity.

Status: Resolved

2. Sell slippage floor is enforced pre-fee and `bps[Sell]` is read at execution, so a fee change retroactively re-prices already-signed authorizations

Severity: Minor

- [ethereum/src/swap/ondo/Ondo.sol:198-214](#)
- [ethereum/src/swap/ondo/Ondo.sol:120-132](#)
- [ethereum/src/swap/Soms.sol:30](#)
- [ethereum/src/swap/Soms.sol:65-68](#)
- [ethereum/src/swap/Soms.sol:72-84](#)

`Ondo.sol` is a custodial broker in front of `Ondo's GMTokenManager`. On `Sell` the user hands over an RWA token and receives a stablecoin less a `CoinList` fee, authorized by an off-chain `CoinList EIP-712` signature (`cSig`) whose `amount` field is a payout floor (`Ondo.sol:43`), enforced at `Ondo.sol:200-202` against the gross balance delta from the manager. The fee is subtracted afterwards at `Ondo.sol:205-206`, so the value the user receives (`output`, transferred at `Ondo.sol:214`) is never compared to `amount`.

The pre-fee ordering is by design: `test/swap/ondo/integration/Ondo.t.sol:165` and `test/swap/ondo/Sell.t.sol:124` both call `setBps(Side.Sell, ...)` before signing, the latter asserting `res == redeemed - fee` at `:160`, and the comment at `Ondo.t.sol:175` reads that "the floor is calculated by us and does not have to be fee aware". The backend signs a gross floor and nets it off-chain.

The defect is the fee being read at execution rather than bound at signing. `bps` reads `bps[Sell]` from storage during the swap (`Soms.sol:65-68`, called from `Ondo.sol:205`). `Soms.setBps` (`Soms.sol:72-84`) is `onlyOwner`, takes effect immediately, has no timelock or ramp limit, and is bounded only by `require(points < ONE_HUNDRED_P)` (`Soms.sol:74`). The rate is bound into neither the `VERIFY_REQUEST` digest (`Ondo.sol:16-18`) nor the Ondo quote, so a fee change landing after signing is absorbed by the user. Measured against the deployed manager on a mainnet fork with `bps[Sell]` raised to 100 after signing, the manager delivered exactly the signed floor of 123460000, yet the user received 122225400, a shortfall of 1234600. `USDonConverter.redeem` is deterministic, so the manager delivers exactly the floor and the payout carries no slack for the fee to come out of.

This is owner-reachable only, and the window is at most one Ondo quote TTL, measured at 32s or less over 6,000 real Ondo trades; an ordinary, non-malicious fee update lands in the same window. The user's RWA is pulled and burned before the fee applies, so the loss is not recoverable, and is bounded per swap by `bps[Sell]`.

Recommendation

We recommend enforcing the signed floor against the value the user actually receives, by moving the check after the fee at `Ondo.sol:200-206`, so that the comparison is made against `val - _fee` rather than against `val`, reverting with `InsufficientAmount()` when the post-fee output falls below the signed amount. Additionally, cap `setBps` (`Soms.sol:74`) at an operational maximum well below `ONE_HUNDRED_P`, and gate fee increases behind a timelock so no authorization can be re-priced inside its own validity window.

This does not bind the Ondo quote into the `CoinList` digest; that is the separate fix recommended in "VERIFY_REQUEST does not commit to the Ondo quote", and neither change subsumes the other.

Status: Acknowledged

The client acknowledges the issue stating that `CoinList`, not the end user, sets the slippage floor, and the fee is already known to the user at signing time through the UI, making retroactive repricing within a validity window unlikely in practice. The client also considers market forces a sufficient cap on fee changes, as excessively high fees would deter usage, and notes that raising fees inside a validity window would carry

reputational consequences that outweigh any fee gain. The client accepts the residual risk given the very short attestation lifespan and the owner-only access control on fee updates.

3. A whitelist entry can become permanently unremovable because `Soms.setInputToken` enforces its `isContract` and `decimals()` guards on the removal path as well as the addition path

Severity: Minor

- [ethereum/src/swap/Soms.sol:86-98](#)
- [ethereum/src/swap/Soms.sol:119-121](#)
- [ethereum/src/swap/ondo/Ondo.sol:32-33](#)
- [ethereum/src/swap/ondo/Ondo.sol:53](#)
- [ethereum/src/swap/ondo/Ondo.sol:123](#)
- [ethereum/lib/shared/Utils.sol:4-12](#)
- [ethereum/src/swap/ISoms.sol:50-52](#)

`Soms.setInputToken(address token, bool val)` (`Soms.sol:86-98`) is the owner's only writer for the `inputTokens` whitelist that gates every swap (`Ondo.sol:53`). It requires `validAddr(token) && isContract(token) (:88)` and `IERC20(token).decimals() <= 18 (:91)` unconditionally, before branching on `val`, so de-whitelisting requires the token to still be well-behaved. A token that later loses its code, or whose implementation changes so that `decimals()` reverts or returns above 18, cannot be removed: every `setInputToken(token, false)` reverts, and `inputTokens` has no other writer. The stale `true` is permanent, leaving only `pause(SWAP_LEVEL)`, `stop()` or a redeploy.

The two guards differ in provenance. `src/swap/TokenSwap.sol`, the older base this contract rewrites, has no `setInputToken` at all. `src/swap/superstate/Superstate.sol:141-149` (out of scope, unchanged since the initial commit) already validated `validAddr && isContract` on the removal path, so that half is carried-forward convention. `Soms.sol:91` adds the `decimals()` guard, which `Superstate` lacks; applying that new guard to the removal path is the in-scope defect, and it is the more fragile of the two, since `decimals()` is an external call into third-party, upgradeable code.

A concrete instance exists. `Ondo.sol:33` writes mainnet USDC into `inputTokens` directly, bypassing `setInputToken`. That address returns `0x` from `eth_getCode` on Base Mainnet, Ethereum Sepolia and Base Sepolia, all listed in the project's deployment table

(ethereum/README.md:53-62). There the entry can neither be used, because a swap reverts at `IERC20(token).decimals()` (Ondo.sol:123), nor cleared, since `isContract(token)` is false (lib/shared/Utils.sol:4-12).

The whitelist is also wider than the venue can service. `setInputToken` admits any deployed contract with `decimals() <= 18`, while the deployed `GMTokenManager` reverts `InvalidDepositToken()` for any deposit that is neither `USDOn` nor mainnet `USDC`. A token can pass every `CoinList` guard, pull the user's funds (Ondo.sol:136), revert with no legible error, and then prove unremovable. Both limbs fail closed, so this is availability and configuration hygiene rather than value loss.

→ [Proof of Concept: P.2](#)

Recommendation

We recommend splitting `setInputToken` so the `isContract` and `decimals()` guards run only when `val == true`; a whitelist entry must always be removable regardless of the token's current state. Separately, take the settlement stablecoin as a constructor argument routed through the same validation instead of hardcoding a mainnet address at Ondo.sol:33, so a deployment cannot start life with an unvalidated, unremovable entry on a chain where that address has no code.

Status: Resolved

4. The CoinList VERIFY_REQUEST digest does not commit to the Ondo quote, so one authorization admits any quote sharing (side, asset, expiration)

Severity: Minor

- [ethereum/src/swap/ondo/Ondo.sol:16-18](#)
- [ethereum/src/swap/ondo/Ondo.sol:57-72](#)
- [ethereum/src/swap/ondo/Ondo.sol:95-112](#)
- [ethereum/src/swap/ondo/Ondo.sol:120-132](#)
- [ethereum/src/swap/ondo/Ondo.sol:176-206](#)
- [ethereum/src/swap/ondo/Types.sol:19-42](#)

`Ondo.swap` (Ondo.sol:46-77) takes an Ondo Quote, Ondo's attestation (`oSig`, forwarded unverified to `GMTokenManager`), and a CoinList EIP-712 authorization (`cSig`), the only access-control root. That digest (Ondo.sol:16-18, :98-107) binds only side, tokens, sender, nonce,

expiry and amount, so of the Quote fields (ondo/Types.sol:19-29) attestationId, quantity and price are unconstrained, and quotes sharing (side, asset, expiration) are interchangeable under one authorization.

On Sell, amount is only a payout floor (minimumReceiveAmount at Ondo.sol:198, require(val >= amount, ...) at :202), never a size cap: Ondo.sol:181 pulls whatever quote.quantity says, against whatever allowance the user granted, with no cap at all. On a mainnet fork, a cSig binding amount = 123460000 pulled 10e18.

On Buy there is no protection. require(val >= quote.quantity) (Ondo.sol:160) is tautological: the manager mints exactly quote.quantity to the caller under nonReentrant with no callback, so the measured delta always equals it and the check cannot fail. Peer contracts' equivalent floor is meaningful only because their expected amount comes from an independently signed order. Ondo.sol:125-132 pins only the product quantity * price to the paid notional, leaving the split bounded only by Ondo's allowedDeviationBps band, live-read as 1000 ($\pm 10\%$): one cSig for amount = 123460000 admitted deliveries spanning 9090909090909090 to 111111111111111111 wei. That band is measured against a reference median 23.0 hours stale across 26 traded assets on-chain, so reachable Buy mispricing exceeds $\pm 10\%$: on a \$100,000 trade, roughly \$11,500 user-adverse at median lag. That requires Ondo's attestation signer to sign off-market; no third party can.

The band is Ondo's, not CoinList's: allowedDeviationBps is settable only by Ondo, capped at 9999. It also composes with reference staleness, which is substantial. Measured on-chain across 26 traded assets, the sanity reference is refreshed on a roughly 38.5 hour cycle, with observed staleness of median 23.0 hours and maximum 38.04 hours, and a median move of 265 bps per refresh. On a \$100,000 Buy that widens the user-adverse envelope from roughly \$9,091 implied by the band alone to roughly \$11,500 at median lag and \$17,473 at the maximum observed lag. Reaching any of it requires Ondo's attestation signer to sign off-market; no third party can.

An on-chain census of GMTokenManager's registered callers yields 13 verified peer integrators; Ondo's own production integrator does not take the attested price on trust: GMTokenLimitOrder (0xf0bc39fc911f6437c84d16188dd8294f7110f451, 26,943 transactions, verified from GMTokenManager's own source tree) enforces a user-authorized limit price on both sides (PriceTooHigh on Buy, PriceTooLow on Sell) and caps the attested size with QuantityTooHigh, annotated as preventing the executor from overspending the user's GM token allowance. Two independent filler teams cap the redeem input leg the same way (RedeemExceedsInput when quote.quantity exceeds the GM supplied), which is the control absent at Ondo.sol:181.

The harm is authorization scope: the signature does not state what it authorizes, so a frontend or backend bug or a leaked cSig succeeds where it should revert.

Ondo.sol:61 and :70 alias req.expiry to quote.expiration; that undocumented aliasing makes the wrapping uint8 nonce (Ondo.sol:141, :185) and the absence of any revocation primitive non-issues, since each would need a stale quote the manager rejects.

Recommendation

We recommend bounding what one authorization admits, in ascending order of cost.

The cheapest change closes the Sell half alone and needs no typehash change: bound the pulled quantity against a value the caller already authorises, rejecting the swap when quote.quantity exceeds it. That is roughly three lines of require at Ondo.sol:180-181, and it is the same shape as the QuantityTooHigh and RedeemExceedsInput guards peer integrators already ship.

Next is an explicit minOut floor on the Buy path: add a signed minOut field to the authorization and enforce require(val >= minOut) against the measured mint delta at Ondo.sol:158-160, symmetric with the Sell path's existing require(val >= amount) at :202. One signed field plus one comparison removes the entire Buy mispricing envelope regardless of what Ondo's reference price does, and it replaces the tautological check at Ondo.sol:160 with a real one.

The complete fix is to commit the CoinList authorization to the quote it authorizes. Add quote.attestationId at minimum, or a full keccak256(abi.encode(quote)) commitment, to the type string at Ondo.sol:16-18, to the VerifyRequest struct at ondo/Types.sol:34-42, and to the abi.encode list at Ondo.sol:98-107, populating it at Ondo.sol:61 and :70. Because attestationId is globally one-shot at the manager, this makes every cSig single-use and non-transferable across quotes in one field. Binding quote.quantity and quote.price as well gives the user-authorized ceiling that Ondo's own integrator enforces.

Separately, record the req.expiry == quote.expiration aliasing as an explicit invariant, via a comment at Ondo.sol:61 and :70, and/or a local require(block.timestamp <= quote.expiration) in swap() so the freshness guarantee is owned by this contract rather than inherited silently from the manager.

This does not bind the fee rate, which is read at execution time; that is the separate fix recommended in "Sell slippage floor is enforced pre-fee", and neither change subsumes the other.

Status: Acknowledged

The client acknowledges the issue stating that the combination of changes made during the audit, existing guards within the contract, Ondo's own protocol-level guards, and the very short time-to-live on each attestation together provide sufficient mitigation. The client considers these layered controls adequate to manage the residual authorization scope risk without implementing the full digest commitment recommended. No additional fixes are planned at this time.

5. Soms.pause accepts a raw bitmask that need not intersect SWAP_LEVEL, so plausible operator values report the contract as Paused while swaps stay fully live

Severity: Minor

- [ethereum/src/swap/Soms.sol:23](#)
- [ethereum/src/swap/Soms.sol:108-110](#)
- [ethereum/src/swap/ondo/Ondo.sol:46-48](#)
- [ethereum/lib/shared/operable/Operable.sol:15-40](#)
- [ethereum/lib/shared/operable/Operable.sol:59-64](#)

Ondo.swap is the only in-scope function guarded by a pause, via `active(SWAP_LEVEL)` (Ondo.sol:48) with `SWAP_LEVEL = 2` (Soms.sol:23). The guard resolves to `Operable._active` (Operable.sol:59-64), which treats the stored paused value as a bitmask and requires that `paused & level` be zero. Only bit 1, the value 2, blocks swaps. `Soms.pause` (Soms.sol:108-110) is the contract's only pause API, takes an unvalidated `uint32`, and `Operable.status()` (Operable.sol:15-29) reports `State.Paused` for any `paused > 0` while `Paused` is emitted unconditionally. Measured on a deployment of the in-scope sources, `pause(1)`, `pause(4)`, `pause(5)` and `pause(8)` each left swaps fully executable while `status()` reported `Paused`; only `pause(2)` and `pause(3)` blocked the swap. `Operable.pause` also overwrites rather than ORs (Operable.sol:35), so setting a second level silently clears the first.

Ordinal arguments are plausible operator choices: neither the parameter name `level` nor the inherited `NatSpec` signals a bit position, and the repo's own `*_LEVEL` constants are ambiguous. `Registry.DEREGISTER_LEVEL` is used solely as a solady role bitmap (Registry.sol:38), whereas `TokenSaleFund.REMIT_LEVEL` is used as both a lifecycle level and a role bitmap on the same function via `started(REMIT_LEVEL)` at TokenSaleFund.sol:142 alongside `onlyRoles(REMIT_LEVEL)` at :143.

The bitmask semantics live in out-of-scope `Operable.sol`, and `TokenSwap.sol:105-107` is a byte-identical pre-existing passthrough. The in-scope surface is the three-line override at `Soms.sol:108-110`, which exposes the raw mask with no validation and no named wrapper. There is no attacker, no on-chain consumer of `status()`, the mistake is correctable in one transaction, and `Swapped` fires on every swap that leaks through.

Recommendation

We recommend validating the mask in `Soms.pause` against the levels the contract honours, e.g. `require(level == 0 || (level & SWAP_LEVEL) != 0, InvalidAmount());`, or expose a named `pauseSwaps(bool)` helper so operators never handle the raw bitmask. At minimum, document in NatSpec that the argument is a bitmask and that only bit 1 affects swaps.

Status: Resolved

6. Value is stranded in the Ondo broker and a standing manager allowance is left behind, because neither input leg reconciles what the manager actually consumed

Severity: Minor

- [ethereum/src/swap/ondo/Ondo.sol:135-138](#)
- [ethereum/src/swap/ondo/Ondo.sol:149-167](#)
- [ethereum/src/swap/ondo/Ondo.sol:180-183](#)
- [ethereum/src/swap/ondo/Ondo.sol:191-214](#)
- [ethereum/src/swap/ondo/Ondo.sol:122-133](#)
- [ethereum/src/swap/Soms.sol:100-106](#)
- [ethereum/src/swap/ISoms.sol:20-21](#)

Ondo is a custodial broker between the user and Ondo's `GMTokenManager`, and it is inconsistent about how it learns how many tokens moved. Both output legs are measured as balance deltas (`Ondo.sol:153, :159` on Buy; `:195, :201` on Sell), and only the measured delta is paid out. Both input legs are credited nominally: `Ondo.sol:136` pulls amount with `trySafeTransferFrom` and then, on the returned boolean alone, approves and forwards `input = amount - fee (:150, :156)`, while `Ondo.sol:181` pulls `quote.quantity` and lets the manager pull that same figure (`:192, :198`). Neither leg compares the broker's own balance before and after the manager call, and neither resets the allowance afterwards. The sibling venue `Superstate` brackets its external-venue call with an exact-spend `require` (`ethereum/src/swap/superstate/Superstate.sol:121-122`); Ondo has no equivalent on either leg.

On the USDC deposit path this is latent: against the deployed manager on a mainnet fork the manager pulls exactly the approved amount, with broker USDC residue and residual allowance both measured at 0, on both Buy and Sell. No user principal is at risk, since the user is always paid from a measured output delta; an under-pull leaves the shortfall in the broker's balance, which holds CoinList's accrued fees.

The live manifestation is the manager's USDon deposit branch, which ignores `depositTokenAmount` and pulls `quote.quantity * quote.price / 1e18` instead. Reaching it takes one owner call, `setInputToken(USDon, true)`, which passes every guard at `Soms.sol:86-98`, and the suite already whitelists an 18-decimal USDon test token (`test/swap/ondo/Buy18Decimal.t.sol:32`, `test/swap/ondo/Sell18Decimal.t.sol:34`). Measured on a mainnet fork, the broker approved the manager for 12346999999999999999, the manager pulled exactly 12346000000000000000, and 9999999999999999 was left stranded in the broker alongside a standing allowance of the same size, recoverable only by the owner through `Soms.transfer` (`Soms.sol:100-106`). The residue accrues one slice per swap, each under the $1e16$ slack at `Ondo.sol:125-132`; that slack also lets the forwarded amount exceed the quote's USD value, and the manager settles the difference as USDon delivered to the broker, never measured by `Ondo.buy`.

Recommendation

We recommend the following changes:

- Mirror the `Superstate.sol:121-122` exact-spend invariant on both input legs: snapshot the input token's balance across the manager call and require that the drop equals the approved amount.
- Call `SafeTransferLib.safeApproveWithRetry(token, manager, 0)` after each manager call so no allowance survives a transaction.
- Forward the USDon refund to `msg.sender`, or account for it separately, so it is not silently retained.
- Apply the same `balanceOf` discipline to the output transfers at `Ondo.sol:167` and `:214`, which today assert only that `safeTransfer` did not revert.
- Add an explicit accrued-fee ledger, for example a `mapping(address => uint256) accruedFees` incremented at `Ondo.sol:147` and `:209` and decremented in `Soms.transfer`, so fee revenue can be distinguished from user-origin residue, making the "input token balance (from fees)" description at `ISoms.sol:20` enforceable rather than aspirational.

These changes bound and account for the residue; they do not change the fact that `Soms.transfer` remains an unrestricted `onlyOwner` sweep of any token to any address.

Status: Partially Resolved

The client partially resolves the issue stating that the residual allowance concern will be remediated by calling `SafeTransferLib.safeApproveWithRetry(token, manager, 0)` after each manager interaction to eliminate any surviving approvals. The stranded value resulting from the USDon deposit branch's imprecise pull (referred to as "Ondo dusting") is accepted as a known limitation. The fix for this finding was combined with a related issue and delivered in a single commit.

7. Ondo.sol declares no status() override, unlike its sibling venue, so the contract reports Active while its Ondo upstream is unusable and every swap reverts

Severity: Informational

- [ethereum/src/swap/ondo/Ondo.sol:136-138](#)
- [ethereum/src/swap/ondo/Ondo.sol:181-183](#)
- [ethereum/src/swap/Soms.sol:43-49](#)
- [ethereum/lib/shared/operable/Operable.sol:15-29](#)
- [ethereum/src/swap/superstate/Superstate.sol:32-54](#)

Ondo.sol inherits Operable.status() (Operable.sol:15-29) unmodified. That implementation reports only the contract's local flags: Stopped if stopped, Paused if paused > 0, Active otherwise. It knows nothing about the upstream venue the contract exists to front, and Soms adds no health view of its own (Soms.sol:43-49).

The point is the in-repo precedent. The sibling swap venue at Superstate.sol:32-54 overrides status() to call super.status(), short-circuit on any non-Active local state, and otherwise fold in the upstream market's state, mapping it to Paused or Stopped. Ondo.sol declares no override at all. So a venue whose upstream can be de-registered, blocklisted, sanctioned, rate-limited, paused by the manager's global or per-token gates, or closed by issuance hours reports Active right up until every swap reverts. No funds are at risk; the cost is that monitoring learns of an outage from the first user complaint rather than from the contract.

Three cheap view reads would cover the main conditions: OndoIDRegistry.getRegisteredID, the compliance check, and the OndoRateLimiter subscription and redemption limit views. The compliance probe must target OndoCompliance at 0x156F73fc73197555e950743Cb2B23F411c751002, whose checkIsCompliant is external view, and not the OndoComplianceGMView shim at 0x54a8757c2FEF8649830b158a8C19D3a670e80318 that the GM tokens resolve through, whose checkIsCompliant is not view and so cannot be called from a view override.

Secondarily, Ondo.sol:136-138 and :181-183 select SafeTransferLib.trySafeTransferFrom and substitute SwapFailed(address,address), discarding the underlying reason, so a blocked end user is indistinguishable from a dead venue. This is not a claim about SafeTransferLib.safeTransfer reverting with a bare TransferFailed() at :167 and :214, which is deliberate library behaviour and not a defect.

Recommendation

We recommend overriding `status()` in `Ondo.sol` following the pattern at `Superstate.sol:32-54`: short-circuit on any locally-set non-Active state, then fold in the upstream view reads (registry registration, compliance, remaining rate-limit capacity) so the venue reports Paused or Stopped when its upstream is unusable. Separately, give the two `trySafeTransferFrom` sites distinct errors, or carry the failing site into `SwapFailed`, so a blocked end user and a dead venue are distinguishable.

Status: Resolved

8. The Swapped event is not self-sufficient because the fee field carried by the sibling ITokenSwap event was deliberately dropped, so no on-chain quantity records what CoinList charged

Severity: Informational

- [ethereum/src/swap/ISoms.sol:11-18](#)
- [ethereum/src/swap/ITokenSwap.sol:9-17](#)
- [ethereum/src/swap/Soms.sol:16](#)
- [ethereum/src/swap/Soms.sol:100-106](#)
- [ethereum/src/swap/ondo/Ondo.sol:170](#)
- [ethereum/src/swap/ondo/Ondo.sol:217](#)

Swapped (`ISoms.sol:11-18`) is the only on-chain record that a CoinList user traded through this venue, and it omits the fee. The sibling interface in the same directory carries it: `ITokenSwap.sol:9-17` declares `Swapped(user, inputToken, outputToken, inputAmount, fee, outputAmount)`. `ISoms.sol:11-18` keeps the same six-field arity but drops fee and appends `uint256 assetPrice`. Both emission sites pass `quote.price` in that final position (`Ondo.sol:170` for Buy, `Ondo.sol:217` for Sell). Nothing anywhere emits the fee. Swapped also omits `Side`, the `data.count` nonce, and `quote.attestationId`.

No emitted argument is factually wrong; both emissions match the transfers that actually occurred, and the defect is omission rather than misstatement. The fee remains recoverable from the emitted amounts and the `bps[side]` in force at that block, but only awkwardly: a consumer must fold the `BpsUpdated` stream (`Soms.sol:79`) with correct intra-block ordering and seed it with the deploy-time default, which is never emitted.

The omission compounds with `Soms.sol:16`, which ships the comment has no global Total storage (TODO: discuss), and with the unbounded `Soms.transfer` (`Soms.sol:100-106`), where the owner may move any amount of any token out with no distinction between accrued fee

revenue and user-origin value. No on-chain quantity anchors a fee sweep.

`assetPrice` is `quote.price` carried from `calldata` into the log. Its only constraint is `OndoSanityCheckOracle.validatePrice`, a deviation band around a possibly stale posted price, so the comment at `ISoms.sol:17` invites reading it as an oracle price.

Recommendation

We recommend adding `uint256 fee` and `Side side` to `Swapped`, and on the `Sell` path emit the pre-fee `val` alongside the post-fee output, so fee revenue, effective price and direction are computable from the single event. Add `uint256 attestationId` so a `CoinList` swap joins Ondo's `TradeExecuted` on a key rather than a transaction hash, and consider adding the `data.count` nonce. Rename `assetPrice` to `quotedPrice` and amend `ISoms.sol:17` to state that it is the attestation's stated price within Ondo's sanity band, not an oracle reading.

Status: Partially Resolved

The client partially resolves the issue stating that the original omissions were a consequence of hitting the stack-too-deep limit, which caused several fields to be squashed out of the event. The client has implemented fixes to emit the fee, side, previous fee value, and attestation ID. The client accepts omitting the post-fee output amount (as it is derivable by subtracting the fee from the pre-fee value) and the count nonce, reasoning that the attestation ID now provides a join key to the sibling `TradeExecuted` event where remaining details can be retrieved.

9. `ondo/Types.sol:11` documents `userId` as binding the end user, while the deployed `GMTokenManager` pins it to the broker, so the comment conceals the complete absence of per-end-user quote binding

Severity: Informational

- [ethereum/src/swap/ondo/Types.sol:6-18](#)
- [ethereum/src/swap/ondo/Types.sol:11](#)
- [ethereum/src/swap/ondo/Ondo.sol:57-76](#)

`Types.sol:11` documents `Quote.userId` as "the user ID the quote is intended for". In this integration that is false, and false in the direction that matters: `userId` binds the quote to `CoinList`'s broker contract, not to any `CoinList` end user. The deployed `GMTokenManager` (mainnet `0x2c158bc4...5c8c`, verified non-proxy source) resolves the caller's registry identity at `GMTokenManager.sol:186` for mint and `:270` for redeem, and `_verifyQuote` then reverts with

`UserIdMismatch` unless it equals `quote.userId` at :336. There `_msgSender()` is the Ondo contract, which holds one registry ID, and registration is permissioned (`OndoIDRegistry.setUserID` is gated on `rwaRole` and `MASTER_CONFIGURER_ROLE`), so per-user IDs cannot be provisioned. Every CoinList user shares one Ondo identity, and Ondo supplies no per-end-user quote binding. `Ondo.sol` never reads `quote.userId`, so nothing in scope depends on it; the defect is entirely in the documentation.

The comment is load-bearing: it is the exact false belief that would lead a reader to conclude that the cross-user consequence of the CoinList EIP-712 digest not committing to the Ondo quote is impossible. All per-user binding must come from that signature (`Ondo.sol:57-76`).

In fairness, `Types.sol:8` instructs that the struct must match Ondo's, and the natspec at `Types.sol:6-18` was copied verbatim from Ondo's own source, including Ondo's `@param` ordering bug that lists `attestationId` before `chainId` while the struct declares `chainId` first. The wording is inherited, not invented, but CoinList's file is where the field's meaning here has to be stated.

Recommendation

We recommend amending `Types.sol:11` to state that `userId` is the broker contract's Ondo registry ID, identical for every CoinList end user, and that it provides no per-end-user binding, since all per-user binding comes from the CoinList EIP-712 signature. While editing the block, reorder the `@param` lines to match the struct's field order.

Status: Resolved

10. The sole on-chain authorization root and the input-token whitelist are written without events, so signer rotation and the deployed whitelist are invisible to log-based monitoring

Severity: Informational

- [ethereum/src/swap/ondo/Ondo.sol:82-85](#)
- [ethereum/src/swap/ondo/Ondo.sol:25-34](#)
- [ethereum/src/swap/ondo/Ondo.sol:111](#)
- [ethereum/src/swap/ISoms.sol:11-27](#)
- [ethereum/src/swap/Soms.sol:36-39](#)
- [ethereum/src/swap/Soms.sol:79](#)
- [ethereum/src/swap/Soms.sol:93](#)
- [ethereum/src/swap/Soms.sol:103](#)

`coinList` is the single on-chain authorization root for the venue: a swap is admitted only if the `CoinList` signature recovers to it (`Ondo.sol:111`), and rotating that address is the only lever that revokes signatures the backend has already issued. `Ondo.setCoinListAddress` (`Ondo.sol:82-85`) changes it and emits nothing; no event for it is declared anywhere in `ISoms.sol`.

The asymmetry is within the same files. `setBps` emits `BpsUpdated` (`Soms.sol:79`), `setInputToken` emits `InputTokenUpdated` (`Soms.sol:93`), `transfer` emits `Transferred` (`Soms.sol:103`), and `Operable` emits `Paused` and `Stopped`. The one owner write that is not logged is the one that changes who may authorize swaps, which is also the single action an attacker holding the owner key would take. Detecting a rotation requires polling the `coinList()` getter or diffing storage; a log-based monitor sees nothing.

The constructors have the same gap and produce an inconsistent log stream rather than a uniformly absent one. `Ondo.sol:25-34` and `Soms.sol:36-39` write `id`, `coinList`, `manager` and `inputTokens[USDC]` silently, while `setInputToken` emits on every later mutation, so an indexer that reconstructs the whitelist by folding `InputTokenUpdated` is wrong from block zero: it reports `inputTokens[USDC] == false` when on-chain it is `true`, and `USDC` is the only token the Buy path accepts. A later `setInputToken(USDC, false)` then emits a transition whose `prev` value has no antecedent in the log stream.

Separately, `setCoinListAddress` omits the `validAddr` check the constructor applies (`Ondo.sol:27`), so setting `address(0)` or `address(this)` bricks every swap, which is a reversible, self-inflicted denial of service, not an authentication bypass, since solady's `ECDSA.recover` reverts on a malformed signature rather than returning `address(0)`.

Recommendation

We recommend declaring a `CoinListUpdated(address prev, address next)` event in `ISoms.sol` alongside `BpsUpdated` and `InputTokenUpdated`, and emit it from both `setCoinListAddress` (`Ondo.sol:82-85`) and the constructor's initial write (`Ondo.sol:30`). Emit the remaining constructor writes as well, namely `id` (`Soms.sol:38`), `manager` (`Ondo.sol:31`) and `InputTokenUpdated(USDC, false, true)` for `Ondo.sol:33`, so the log stream is complete from block zero. Add `require(validAddr(addr), InvalidAddress())` to `setCoinListAddress`.

Status: Partially Resolved

The client partially resolve the issue stating that they will add a `CoinListUpdated` event and emit it from both the setter and constructor, and will also emit `InputTokenUpdated` from the constructor to close the log-stream gap for USDC. However, they accept the absence of events for `id` and `manager`, noting that `manager` is immutable in practice and `id` is a non-critical internal field handled consistently with prior contracts. The fix is implemented in commit `3c821b8`.

11. Conventions carried forward from the pre-existing out-of-scope swap venues into Ondo/Soms: `renounceOwnership`, an irreversible `stop()`, a constructor-bypassed whitelist write, and an inert `onlyOwner` gate

Severity: Informational

- [ethereum/src/swap/Soms.sol:3](#)
- [ethereum/src/swap/Soms.sol:112-114](#)
- [ethereum/src/swap/ondo/Ondo.sol:32-33](#)
- [ethereum/src/swap/ondo/Ondo.sol:88-90](#)
- [ethereum/src/swap/ondo/Ondo.sol:228-231](#)

Each item below is byte-identical or near-identical to code shipped in the out-of-scope initial commit `ffb7137`, so these are conventions carrying forward, not defects this pull request introduced. Each has a one-line fix.

`Soms` inherits `solady's Ownable unoverridden` (`Soms.sol:3`), as `TokenSwap.sol:13` does. `renounceOwnership()` zeroes the owner in one unconfirmed call, and since swap is gated on `active(SWAP_LEVEL)` rather than ownership, the result is a live but unadministrable venue: `transfer`, `stop`, `pause`, `setBps`, `setInputToken` and `setCoinListAddress` become permanently unreachable while `Ondo.swap` keeps working. Fix: override `renounceOwnership()` to revert.

`Soms.stop` (`Soms.sol:112-114`) is byte-identical to `TokenSwap.sol:109-111` and delegates to `Operable.stop`, which sets `stopped = true` with no un-stop. Every plausible Ondo-side trigger, whether a token pause, a stale price, an exhausted rate limit or thin liquidity, is transient, while `stop()` is not. Fix: document `pause(SWAP_LEVEL)` as the response to a transient upstream outage and `stop()` as a decommission.

`Ondo.sol:32-33` writes mainnet USDC into `inputTokens` directly, bypassing the `validAddr && isContract` and `decimals() <= 18` checks `setInputToken` (`Soms.sol:86-98`) enforces elsewhere; `Superstate.sol:26-27` hardcodes the same address the same way. The related defect, that `setInputToken`'s guards run on the removal path and make such an entry unremovable, is reported separately. Fix: take the address as a constructor argument and route it through that validation.

`Ondo.domainSeparator()` (`Ondo.sol:88-90`) is `onlyOwner`, but the gate restricts nothing: `solady's public eip712Domain()` lets anyone reconstruct the separator, which is public knowledge anyway. The cost is the misleading comment at `Ondo.sol:228`, asserting an access-control effect that does not exist, plus tests needing the owner role to build signatures. Fix: make it plain `public view` and drop the `_domainSeparator()` override and comment.

Recommendation

We recommend overriding `renounceOwnership()` to revert; document `pause(SWAP_LEVEL)` as the response to transient upstream outages and `stop()` as a decommission; pass the input token address as a constructor argument and validate it through the same checks `setInputToken` applies; and remove the `onlyOwner` modifier from `domainSeparator()` together with the `_domainSeparator()` override and its comment.

Status: Resolved

Proof of Concept

P.1 — Permanent denial of service of a (user, input, output) swap pair once the packed uint72/uint88 OptimizedSwapTotal accumulators saturate, as they are checked-add and have no reset path

← [Back to finding_1](#)

An 18-decimal input token at `bps[Buy] = 100` accrues `amount * 100 / 10100` per swap. Four buys of `100_000e18` record `feeSum = 3960.396e18`; the fifth would take it to `4950.495e18`, above the uint72 maximum of `4722.366e18`, and reverts with `Panic(0x11)`. The configuration mirrors `ethereum/test/swap/ondo/Buy18Decimal.t.sol`.

```
function testFeeSumOverflowBricksUser() public {
    assert(ondo.setBps(Side.Buy, 100)); // 1%

    uint256 amt = 100_000e18;
    inT18.mint(ALICE, 1_000_000e18);
    vm.prank(ALICE);
    inT18.approve(address(ondo), type(uint256).max);

    (, uint256 input) = ondo.fee(Side.Buy, amt);
    a.setPrice(1e18);
    a.setQuantity(input);
    Quote memory q = a.quote(block.chainid, Side.Buy);

    for (uint64 i = 0; i < 4; i++) {
        _mockManager(q);
        vm.prank(ALICE);
        ondo.swap(q, bytes("x"), address(inT18), amt, _sig(address(inT18),
q.asset, i, amt));
    }

    // 5th swap: feeSum would exceed type(uint72).max
    _mockManager(q);
    vm.expectRevert(stdError.arithmeticError); // Panic(0x11)
    vm.prank(ALICE);
    ondo.swap(q, bytes("x"), address(inT18), amt, _sig(address(inT18), q.asset,
4, amt));
}
```

The reverting call leaves the packed slot byte-identical, so the failure is a clean revert with no carry into an adjacent field. The same saturation occurs on the Buy path when the 18-decimal input token is USDOn.

P.2 — A whitelist entry can become permanently unremovable because Soms.setInputToken enforces its isContract and decimals() guards on the removal path

as well as the addition path

← [Back to finding 3](#)

Against the real GMTokenManager on a mainnet fork at block 25903119, DAI passes setInputToken and every subsequent CoinList check, and the Buy then reverts inside Ondo with an unnamed custom error raised three contracts deep.

```
function test_F_NonUsdcInputReverts() public {
    address DAI = 0x6B175474E89094C44Da98b954EedeAC495271d0F;
    ondo.setInputToken(DAI, true);
    uint256 p = postedPrice(HOODON);
    uint256 qty = 1e18;
    uint256 amount = qty * p / 1e18; // DAI is 18dp
    deal(DAI, user, amount);
    vm.prank(user);
    IERC20(DAI).approve(address(ondo), type(uint256).max);
    Quote memory q = mkQuote(Side.Buy, HOODON, p, qty);
    bytes memory c = cSig(Side.Buy, DAI, HOODON, user, 0, q.expiration, amount);
    vm.prank(user);
    vm.expectRevert();
    ondo.swap(q, oSig(q), DAI, amount, c);
}
```